

ТЕМА: ЛИНЕЙНЫЙ ПОИСК И СЛОЖНОСТИ АЛГОРИТМОВ



ВВЕДЕНИЕ

Для работы с любым языком программирования требуется знать алгоритмы, именно в этой теме мы начинаем их изучение. И начинаем мы с основных понятий и примеров

УРОК: ЛОГАРИФМ

Логарифм

Функция, обратная экспоненциальной функции. Используется для описания скорости роста алгоритмов, таких как двоичный поиск



Логарифмическое время выполнения ($O(\log n)$)

Применяется к алгоритмам, которые уменьшают размер проблемы в каждой итерации, например, двоичный поиск

Пример кода

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1 # Если элемент не найден

# Пример использования:
arr = [10, 20, 30, 40, 50]
target = 30
print("Индекс элемента:", binary_search(arr, target))
```

Объяснение

Логарифмическое время: При каждом шаге количество возможных элементов для поиска уменьшается вдвое, что приводит к $O(\log n)$ сложности

ВЫВОД

Мы охватили основы алгоритмов, а также важность логарифмов в анализе сложности алгоритмов. Эти концепции являются основой для понимания более сложных алгоритмов и структур данных, которые мы будем изучать в дальнейшем

